

Curso de Go

Aula 12 - Contextos e cancelamento
Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 `código_`
BRAZUCA ^[3]





Nessa Aula nos Vamos Ver

- **O que é o pacote context**
- **Como compartilhar contexto entre funções e goroutines**
- **Uso de timeout e deadlines**
- **Cancelamento gracioso de operações**
- **Encerramento seguro de processos concorrentes**
- **Comunicação de cancelamento entre goroutines**
- **Boas práticas com context.Context**
- **Parte Prática**



O que é context.Context

- O pacote context é utilizado para:
- Compartilhar sinais de cancelamento
- Definir limites de tempo
- Passar informações entre chamadas
- Controlar operações concorrentes
- O contexto normalmente é utilizado em:
 - APIs
 - Bancos de dados
 - Goroutines
 - Sistemas distribuídos
 - Requisições HTTP
- O contexto é propagado entre funções.



Por que context é importante

- **Uma goroutine pode continuar executando mesmo quando não é mais necessária.**
- **Problemas possíveis:**
 - **Vazamento de goroutines**
 - **Consumo excessivo de memória**
 - **Processamento desnecessário**
 - **Timeout infinito**
 - **Recursos presos**
- **O context resolve isso permitindo:**
 - **Cancelamento centralizado**
 - **Controle de tempo**
 - **Encerramento organizado**
- **Muito importante em sistemas concorrentes.**



Cancelamento com `context.WithCancel`

- `context.WithCancel()` permite cancelar manualmente uma operação, enviando um sinal para todas as goroutines e funções que utilizam aquele contexto.
- Quando a função `cancel()` é chamada, o channel retornado por `ctx.Done()` é fechado automaticamente, permitindo que as goroutines encerrem de forma segura e organizada.
- É muito utilizado para cancelamento gracioso de tarefas concorrentes, evitando vazamento de goroutines, consumo desnecessário de recursos e processos executando sem necessidade.



Exemplo context.WithCancel()

```
package main

import (
    "context"
    "fmt"
    "time"
)

func tarefa(ctx context.Context) {

    for {

        select {

            case <-ctx.Done():
                fmt.Println("Tarefa cancelada")
                return

            default:
                fmt.Println("Executando...")
                time.Sleep(time.Second)
            }
        }
    }

}

func main() {

    ctx, cancel := context.WithCancel(context.Background())

    go tarefa(ctx)

    time.Sleep(3 * time.Second)

    cancel()

    time.Sleep(time.Second)
}
```




Entendendo ctx.Done()

- O `ctx.Done()` retorna um channel.
- Quando o contexto é cancelado: O channel é fechado automaticamente.
- Benefícios:
 - Encerramento gracioso
 - Liberação correta de recursos
 - Evita processos presos
- Exemplo:

```
<-ctx.Done()
```

- Isso faz a goroutine esperar cancelamento.



Timeout com `context.WithTimeout`

- **`context.WithTimeout()` cria um contexto com tempo máximo de execução, cancelando automaticamente a operação quando o limite definido é atingido.**
- **É muito utilizado em chamadas de API, acesso a banco de dados e operações demoradas, evitando que processos fiquem presos indefinidamente.**



Exemplo Timeout com context.WithTimeout

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {

    ctx, cancel := context.WithTimeout(
        context.Background(),
        3*time.Second,
    )

    defer cancel()

    <-ctx.Done()

    fmt.Println("Tempo esgotado")
}
```



Deadlines com context.WithDeadline

- **Deadlines com context.WithDeadline**
- **Muito útil em:**
 - **Sistemas distribuídos**
 - **Controle global de execução**
 - **Processamento sincronizado**
- **Exemplo**

```
deadline := time.Now().Add(5 * time.Second)
```

```
ctx, cancel := context.WithDeadline(  
    context.Background(),  
    deadline,  
)
```

```
defer cancel()
```



Cancelamento gracioso de goroutines

- **Cancelamento gracioso de goroutines significa encerrar tarefas concorrentes de forma segura, permitindo finalizar operações importantes antes da parada completa.**
- **Utilizando `context.Context`, as goroutines conseguem detectar sinais de cancelamento através do `ctx.Done()` e interromper sua execução organizadamente.**
- **Esse modelo evita vazamento de goroutines, corrupção de dados e consumo desnecessário de memória e processamento.**
- **Na prática**
 - **Encerrar sem corrupção de dados**
 - **Finalizar tarefas pendentes**
 - **Liberar memória corretamente**



Exemplo cancelamento gracioso de goroutines

```
package main

import (
    "context"
    "fmt"
    "time"
)

func worker(ctx context.Context) {

    for {

        select {

            case <-ctx.Done():
                fmt.Println("Worker encerrado")
                return

            default:
                fmt.Println("Processando")
                time.Sleep(time.Second)
            }
        }
    }

}

func main() {

    ctx, cancel := context.WithCancel(context.Background())

    go worker(ctx)

    time.Sleep(5 * time.Second)

    cancel()

    time.Sleep(time.Second)
}
```



Passando valores com context.WithValue

- **context.WithValue()** permite compartilhar informações entre funções e goroutines utilizando o próprio contexto, sem precisar criar variáveis globais.
- É muito utilizado para transportar dados como ID de requisição, usuário autenticado, tokens e metadados durante o fluxo da aplicação.
- O uso deve ser moderado, pois o principal objetivo do context.Context é controle de cancelamento e timeout, não armazenamento genérico de dados.
- Muito usado para:
 - IDs de requisição
 - Usuário autenticado
 - Tokens
 - Metadados



Exemplo passando valores com context.WithValue

```
package main

import (
    "context"
    "fmt"
)

func main() {

    ctx := context.WithValue(
        context.Background(),
        "usuario",
        "Antonio",
    )

    nome := ctx.Value("usuario")

    fmt.Println(nome)
}
```



Boas práticas com context.Context

- **Sempre propague o contexto**
 - `func processar(ctx context.Context)`
- **Nunca ignore cancelamentos**
- **Sempre verificar:**
 - `ctx.Done()`
- **Use defer cancel()**
 - `defer cancel()`
- **Evita vazamento de recursos.**
- **Não armazenar context em structs**
- **Passe como parâmetro.**
- **Context normalmente é o primeiro argumento**
 - `func salvar(ctx context.Context)`
 - **Evite usar WithValue excessivamente: Use apenas dados realmente necessários.**
 -

The screenshot shows a Go IDE with a file named 'package main' in 'Untitled-1'. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then uses pointers to call these methods. Comments in Portuguese explain the pointer receiver and the automatic conversion between values and pointers for method calls.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Contextos e cancelamento